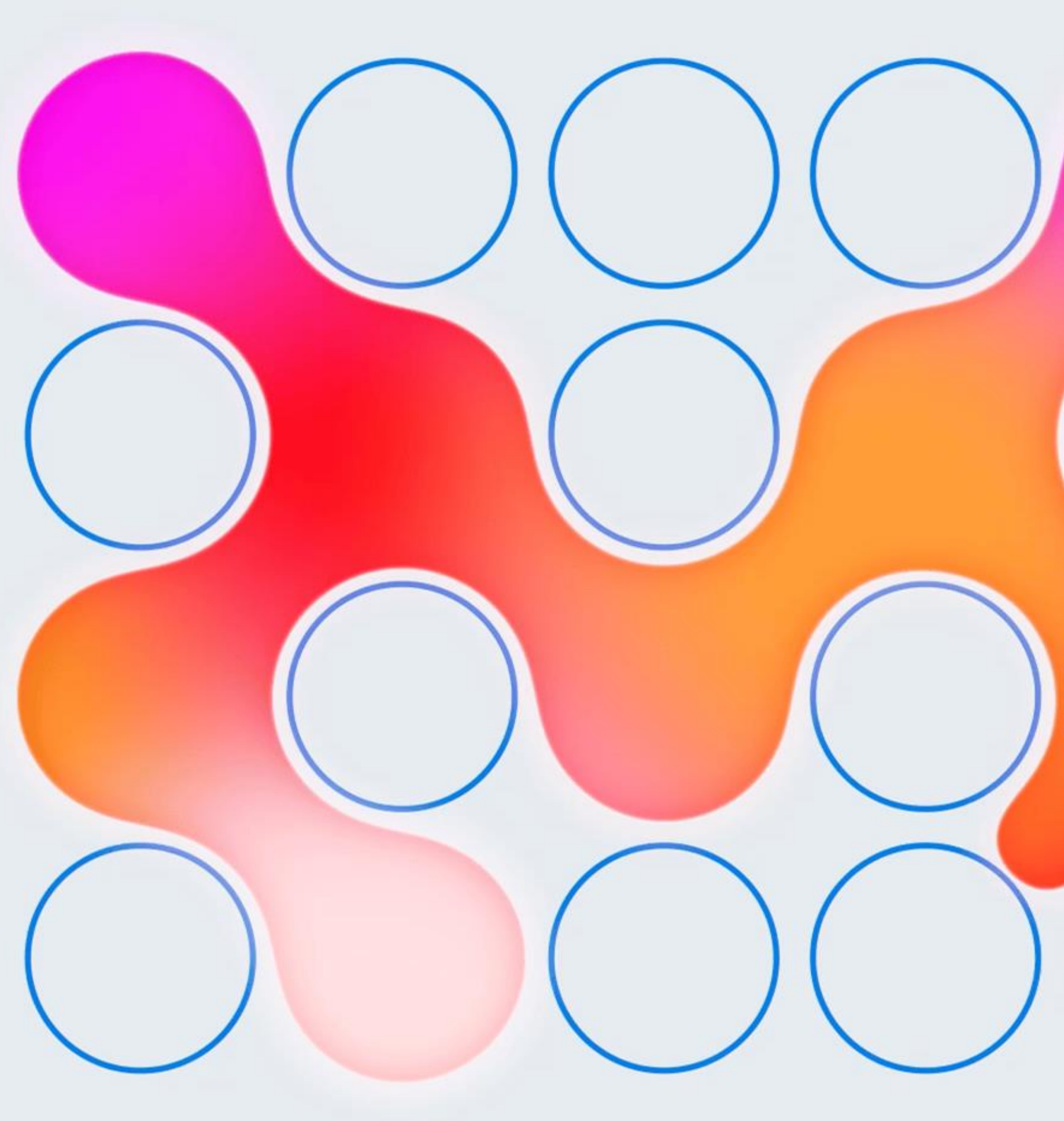




Building Agentic Systems on AWS

Agent evaluation and decision engines



Presenter

Leonardo Murillo
Developer Relations,
AWS Marketplace



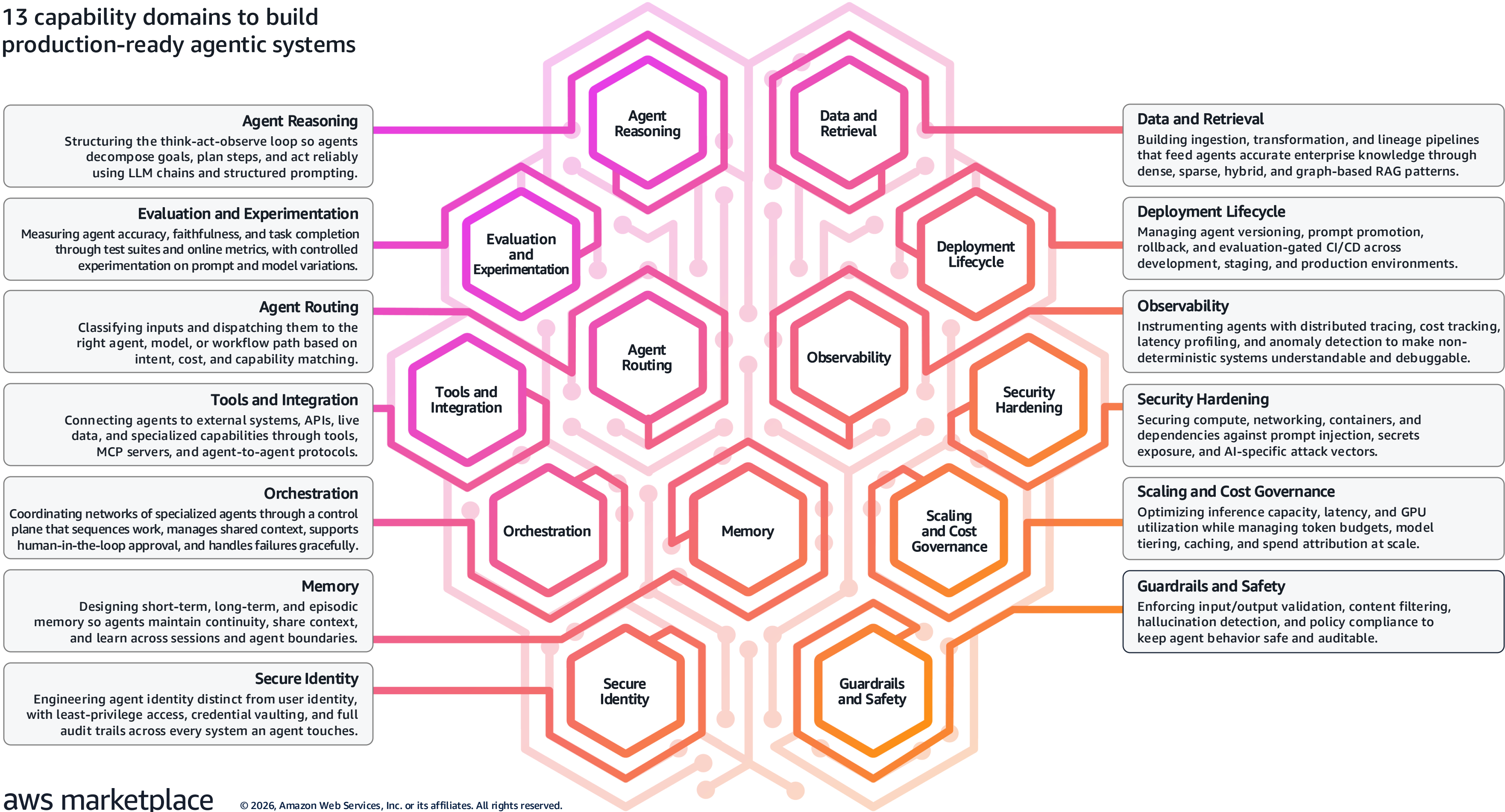
Agenda

- The four evaluation dimensions
- Building an evaluation pipeline on AWS
- Test dataset design and failure pattern analysis
- Evaluation-driven development cycle
- LLM-as-judge: Scalable automated scoring
- Routing agents and decision patterns
- AI tool landscape
- Demo
- Next steps



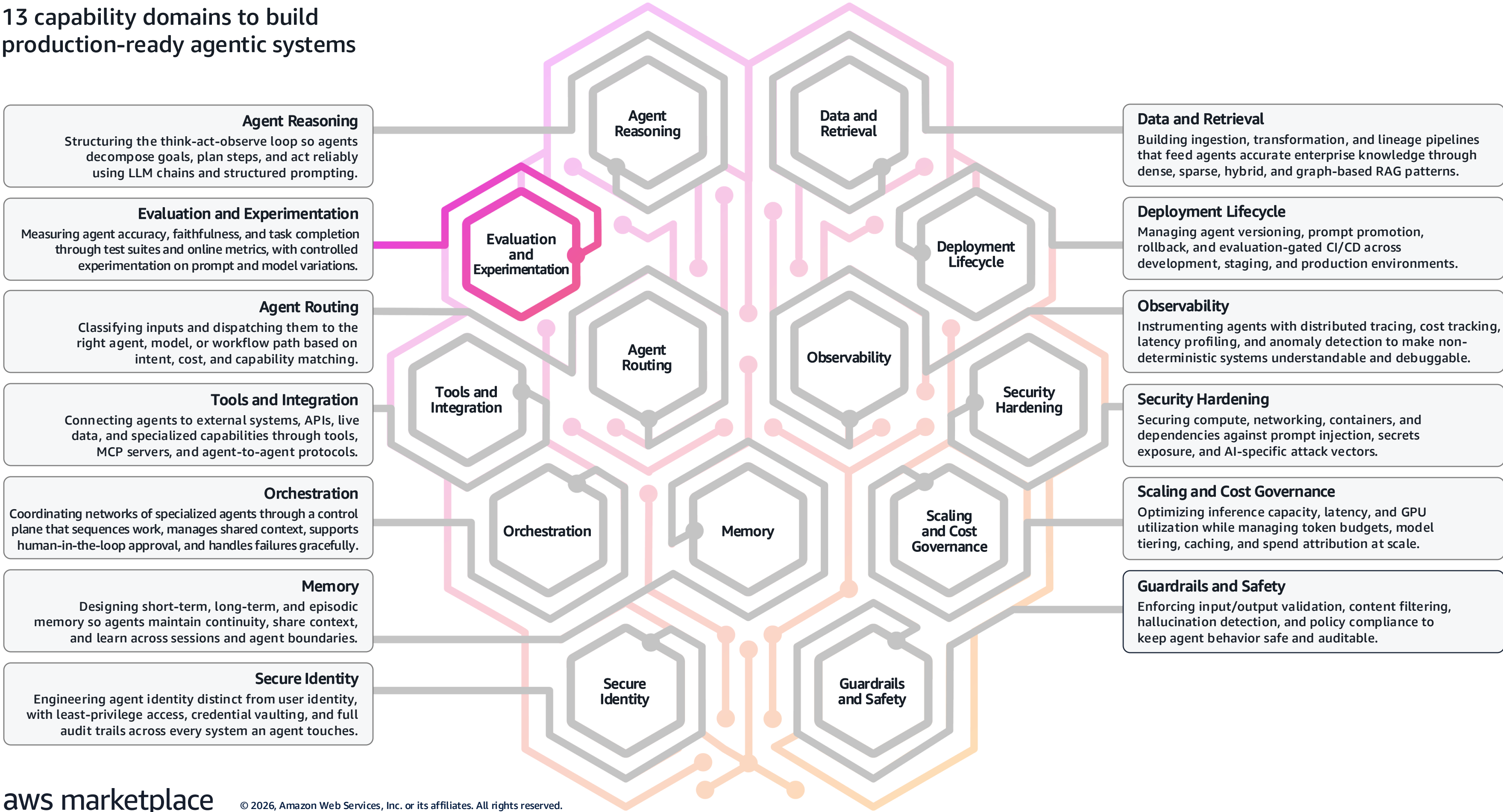
The Agentic AI capability stack

13 capability domains to build production-ready agentic systems



The Agentic AI capability stack

13 capability domains to build production-ready agentic systems



Agentic AI tools and capabilities in AWS Marketplace

AGENT REASONING

Claude

cohere

Hugging Face

LangChain

Dify

Progress Agentic RAG

composable

Amazon Bedrock

STRANDS AGENTS

EVALUATION & EXPERIMENTATION

LangChain

Langfuse

vellum

Weights & Biases

comet LLM

Patronus AI

deepchecks.

Amazon Bedrock AgentCore (Evaluations)

Amazon SageMaker AI

Amazon CloudWatch

AGENT ROUTING

LangChain

AWS Step Functions

Amazon EventBridge

Amazon Bedrock

TOOLS & INTEGRATION

algolia

mapbox

Deepgram

Kong Connect

Tyk

TAILWINDS

portkey

SMARTBEAR

krakenD

AWS Lambda

Amazon API Gateway

Amazon Bedrock AgentCore (Gateway)

ORCHESTRATION

Temporal

orkes

PREFECT

ASTRONOMER

AWS Step Functions

Amazon Bedrock Agents

Amazon MWAA

MEMORY

zilliz

redis

elastic

TiDB Cloud

MongoDB Atlas

Weaviate

yugabyteDB

Pinecone

Amazon OpenSearch

Amazon MemoryDB

Amazon Bedrock AgentCore (Memory)

SECURE IDENTITY

Ping Identity

okta

CYBERARK

Amazon Cognito

AWS Secrets Manager

Amazon Bedrock AgentCore (Identity)

DATA & RETRIEVAL

CONFLUENT

databricks

Fivetran

Collibra

Alation

Bigeye

MONTE CARLO

boomi

Amazon Bedrock Knowledge Bases

Amazon S3

AWS Glue

DEPLOYMENT LIFECYCLE

Pulumi

GitLab

Vercel

LaunchDarkly

Vault

KNIME

AWS CodePipeline

Amazon SageMaker AI

AWS CDK

OBSERVABILITY

DATADOG

honeycomb.io

arize

LangWatch

True Watch

INSTANA

logz.io

Amazon CloudWatch

AWS X-Ray

AWS Distro for OpenTelemetry

SECURITY HARDENING

snyk

Protect AI

Prompt

tailscale

Amazon GuardDuty

AWS Security Hub

Amazon Inspector

SCALING & COST GOVERNANCE

ClickHouse

NVIDIA AI ENTERPRISE

Modal

cast.ai

baseten

anyscale

hydrolix

StormForge

Amazon EKS

AWS Fargate

Amazon Bedrock AgentCore (Runtime)

GUARDRAILS & SAFETY

Claude

Guardrails AI

Arthur

Amazon Bedrock Guardrails

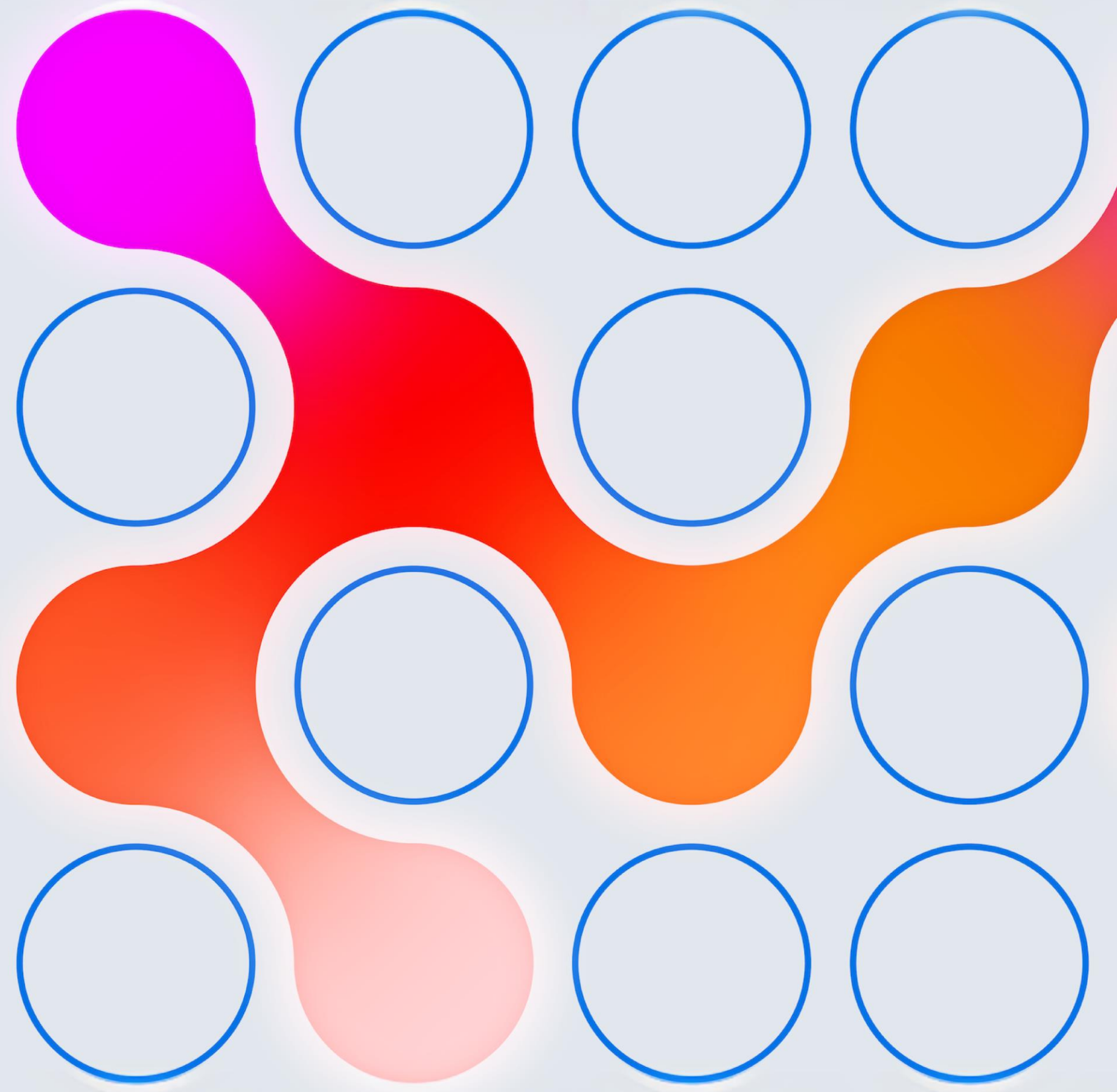
Amazon Bedrock AgentCore (Policy)

2,800+

Agentic AI tools and capabilities

To help you build, deploy, and operate agentic systems on AWS — available through your AWS account.

The four evaluation dimensions



The four evaluation dimensions

1

Task completion

Did the agent fully achieve the stated goal? The most critical signal — binary pass/fail or graded partial credit.

2

Reasoning quality

Did the agent take a sound path? An agent that reaches the right answer through hallucinated intermediate steps fails on novel inputs.

3

Operational metrics

Is the agent practical to run? Latency, cost per task, tool call count, and loop iterations tell you whether the agent is efficient.

4

Safety and reliability

Does the agent behave within defined boundaries? Guardrail violations, policy adherence, error recovery, and timeout behavior.

Task completion: Measure it right

Three approaches, by task complexity:

Binary pass/fail

Recommended for tasks with a single correct output — e.g., "did the CDK stack deploy?". Deterministic, cheap, fast to run.

Partial credit scoring

Score 0–5 on defined criteria when many valid outputs exist. Requires a rubric and an LLM judge. Use for open-ended report generation.

Human spot check

Essential for calibrating the large language model (LLM) judge. Periodically sample 10% of outputs and have a domain expert score them. Use the divergence to detect judge drift.

What makes a good test dataset:

● Input variety

Cover all expected task types, including edge cases. A dataset of only happy-path examples miss 80% of real failures.

● Adversarial prompts

Include inputs designed to confuse the agent: ambiguous goals, missing context, contradictory instructions.

● Expected output spec

Define expected outputs in terms of conditions, not exact strings: "CDK stack includes Amazon ECS" not "output equals X".

● Start small, grow deliberately

Begin with 20 to 50 high-quality examples. Add new cases every time you discover a failure mode in production.



Recommendation: Patronus AI (AWS Marketplace) excels at custom evaluation criteria and regression tracking across agent versions—ideal when your output format evolves.

Reasoning quality: Inspecting agent traces

1 Correct tool sequence

Did the agent invoke tools in the right order? Wrong sequence often means the agent is guessing, not reasoning.

2 Evidence-grounded reasoning

Does the agent's reasoning text account for the tool results it received? Ignoring results is a hallucination precursor.

3 Valid tool parameters

Were arguments well-formed and sensible? Invalid parameters reveal prompt ambiguity in the tool description.

4 No unsupported claims

Are all factual assertions traceable to context? Claims that can't be found in the trace are hallucinations.

▶ **Tools for trace inspection at scale:** LangSmith (LangChain) and LangFuse both provide trace UIs that make per-step reasoning inspection practical. Deepchecks (AWS Marketplace) automates hallucination detection in your pipeline.

Operational metrics: Running agents efficiently

Metric	What it tells you	How to capture on AWS	Target / alert
Latency (p50/p90/p99)	Total time from goal submission to final response. p99 is the number that matters for user experience.	AWS Lambda duration; emit custom metric per task type	< 10 s (p95)
Cost per task	Total input + output token cost for a complete task execution. Track across versions—prompt changes that improve quality often increase cost.	Amazon Bedrock model invocation cost tags in the Cost Explorer	Set a budget; alert on +20% increase
Tool call count	Average number of tool invocations per task. High counts signal a confused agent or an ambiguous tool description.	Emit custom Amazon CloudWatch metric at task completion	≤ 8 per task (tune to your agent)
Loop iterations	Number of think-act-observe cycles. Runaway loops—far exceeding your expected max—indicate a stuck agent.	Count iterations in the Lambda handler; emit metric	Alert at 2× expected max

 **Recommendation:** Emit all four metrics as CloudWatch custom metrics with dimensions for task_type and agent_version. This lets you drill down by capability and detect regressions per task category, not just globally.

Safety and reliability: Behavioral boundaries

Guardrail violations

How many requests were blocked or modified by Amazon Bedrock Guardrails? A spike signals a shift in input distribution — possibly adversarial prompts entering your system.

Amazon Bedrock Guardrails — built-in metrics in CloudWatch

Error recovery rate

When a tool call fails, does the agent recover and complete the task, or does it give up? An agent with low error recovery becomes a reliability liability at scale.

Instrument try/except in tool handlers; emit success/failure metric

Policy adherence rate

What fraction of task executions completed without attempting an action outside the agent's authorized scope? Measure by checking tool call logs against an allowed action list.

Custom CloudWatch metric; emit per task with outcome label

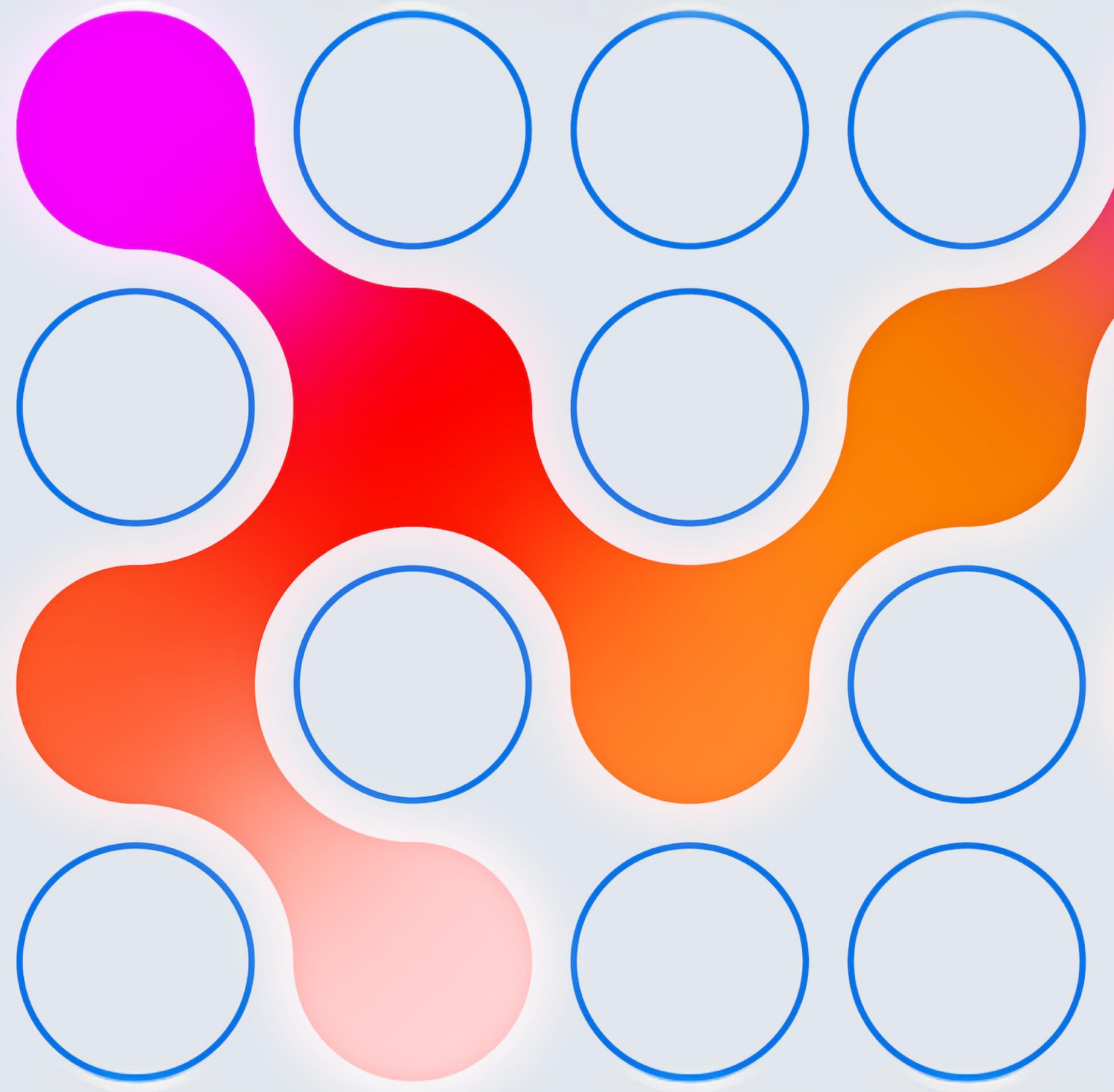
Timeout behavior

When the agent reaches its maximum iteration count without completing the task, does it produce a useful partial result or a confusing error? Define a graceful degradation policy explicitly.

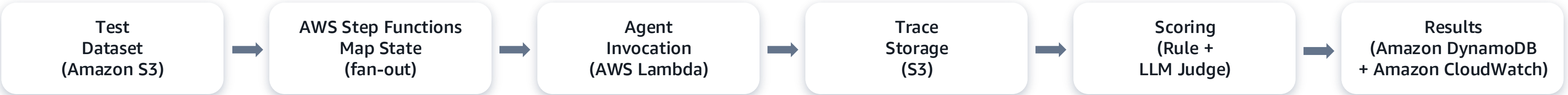
Set max_iterations in the agent loop; emit timeout metric to CloudWatch

▶ **AWS recommendation:** Configure Amazon Bedrock Guardrails before your first production deployment—not as an afterthought. Content filtering, topic restrictions, and PII redaction are all available natively.

Building an evaluation pipeline on AWS



Evaluation pipeline architecture on AWS



Step Functions state machine

- **Map state (fan-out)** Parallel execution of all test cases
- **Max concurrency** Set to avoid throttling Amazon Bedrock API limits
- **Catch + retry** Handle transient agent failures gracefully
- **Fan-in → scoring** Aggregate traces, then invoke the judge step

Two-layer scoring strategy

- Layer 1 — Rule-based (fast, deterministic)**
- Schema validation, token budget check, required output components present. No LLM cost. Runs first and gates LLM scoring.
- Layer 2 — LLM-as-judge (nuanced, scalable)**
- Send output + expected + rubric to a separate model. Use a different model than the agent to avoid self-evaluation bias.
- Only run LLM scoring on cases that pass rule-based checks. This cuts judge cost by 30–50% on typical agent workloads.

Designing your test dataset

Three required components

Input goals

The tasks you want the agent to complete. Cover the full range — common inputs, edge cases, and adversarial prompts. Not just the happy path.

Expected outputs

For each input: what does a correct response look like? Can be a specific artifact, a report format, or a set of conditions the output must satisfy.

Evaluation criteria

Per input/output pair: the rubric used to judge the agent. Some criteria are binary (pass/fail); others are scored 1 to 5.

Five input categories to cover

- **Basic cases**
Single service, straightforward dependencies—your happy path baseline
- **Complex multi-service**
Service-to-service dependencies that require inference from context
- **Existing IaC**
Repos with infrastructure-as-code that the agent must read and extend
- **Ambiguous config**
Conflicting or incomplete configuration— forces the agent to choose
- **Security edge cases**
Exposed credentials, overly broad IAM policies, missing encryption

Our recommendations

Start with 20–50 examples

Small and high-quality beats large and noisy. Expand as you discover failure modes—every production incident can become a test case.

Weight your dataset deliberately

Don't sample inputs uniformly. Overweight the failure modes you've already seen. Rare but critical inputs (security, data loss) deserve 3-5× representation.

Version your dataset in S3

Your dataset is code. Tag each version with the agent version it was designed against. Use Vellum or DynamoDB for structured versioning with diff tracking.

Test for what can go wrong, not what usually works

If 90% of your inputs are clean and well-formed, you have a 90% pass rate with an agent that fails on everything real users throw at it.

Reading failures: Turn bad results into real improvements

A failing test case is not the problem — it is the signal. The pattern across failures tells you where to fix.

Pattern: Same tool fails 80%+ of the time

Diagnosis: The tool's description is ambiguous or its schema is unclear.

Fix: Rewrite the tool description. Add a concrete example of valid and invalid inputs. The agent uses tool descriptions as its only guide to parameter values.

Pattern: Failures cluster on one input class

Diagnosis: The system prompt doesn't handle this input class explicitly.

Fix: Add explicit instructions for the failing input class. Include examples in the prompt. Consider a separate specialist agent if the class requires fundamentally different reasoning.

Pattern: High tool call count on failing tasks

Diagnosis: The agent is confused—looping and retrying instead of reasoning to a conclusion.

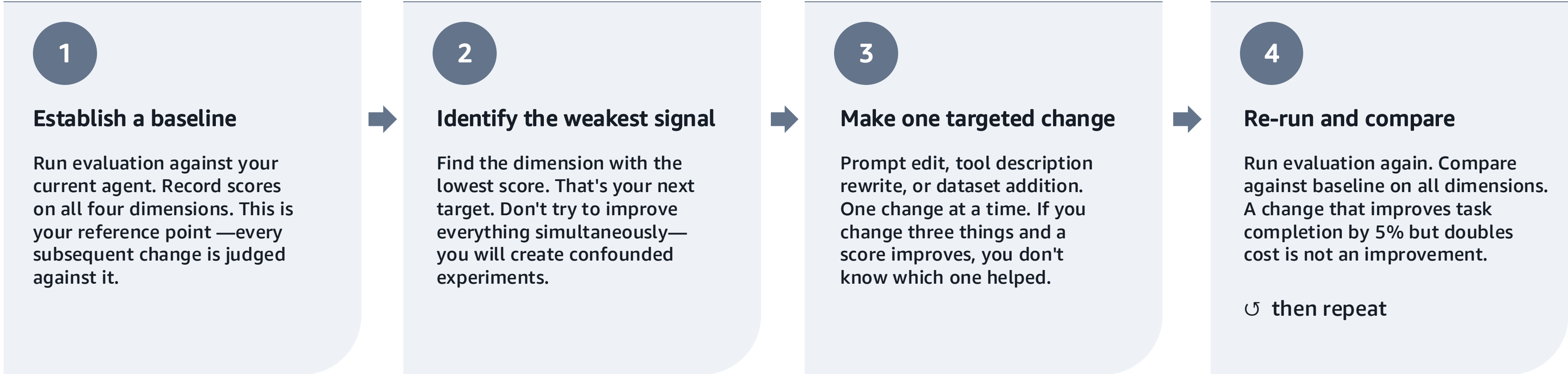
Fix: Check whether the agent is receiving contradictory information from different tools. Reduce tool overlap. Add an explicit stop condition to the system prompt.

Pattern: Failures appear random, no clear pattern

Diagnosis: The agent is operating near its capability boundary, or the task definition is underspecified.

Fix: Inspect 10 individual traces manually. Look for the point where reasoning diverges. Either decompose the task further or add more examples to the system prompt.

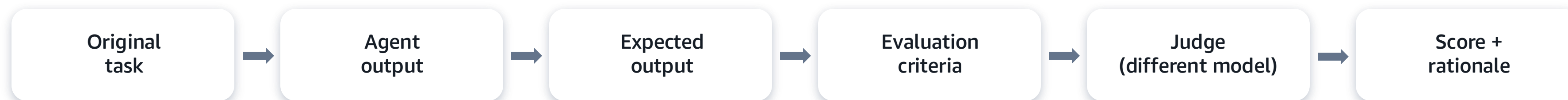
Evaluation-driven development: The improvement cycle



Real numbers — DevOps Companion baseline vs. after one targeted prompt fix:

Metric	Baseline	After fix	What changed
Task completion rate	78%	89%	Prompt: explicit Dockerfile base image instruction
Completeness score	80%	88%	Added container orchestration inference rule
Actionability score	60%	85%	Largest gain was the targeted dimension
Avg tool call count	6.2	5.8	Fewer retries after clearer instructions

LLM-as-judge: Design best practices for quality results



Critical design rules

Use a different model as judge

Self-evaluation bias is real. Claude 4.6 Sonnet as agent + Claude 4.6 Opus as judge is a proven pairing. Never use the same model version for both.

Ground the rubric in observable evidence

Criteria like "is this good?" produce inconsistent scores. Criteria like "does the output include a valid VPC CIDR block?" produce reliable ones.

Include a rationale requirement

Ask the judge to explain its score in 2–3 sentences. A rationale reveals when the judge is confused, which pure scores cannot.

Periodically calibrate with human scores

Run human spot-checks on 10% of judge outputs quarterly. Calibrate your rubric when human and judge scores diverge by more than 15%.

Example rubric: Reasoning quality

5

All tool calls used correct parameters. Reasoning accounts for every tool result. No unsupported factual claims.

4

Minor parameter issue in ≤ 1 tool call. Reasoning mostly accounts for results. ≤ 1 unsupported claim.

3

Parameter issues in ≤ 2 calls or reasoning partially ignores results.

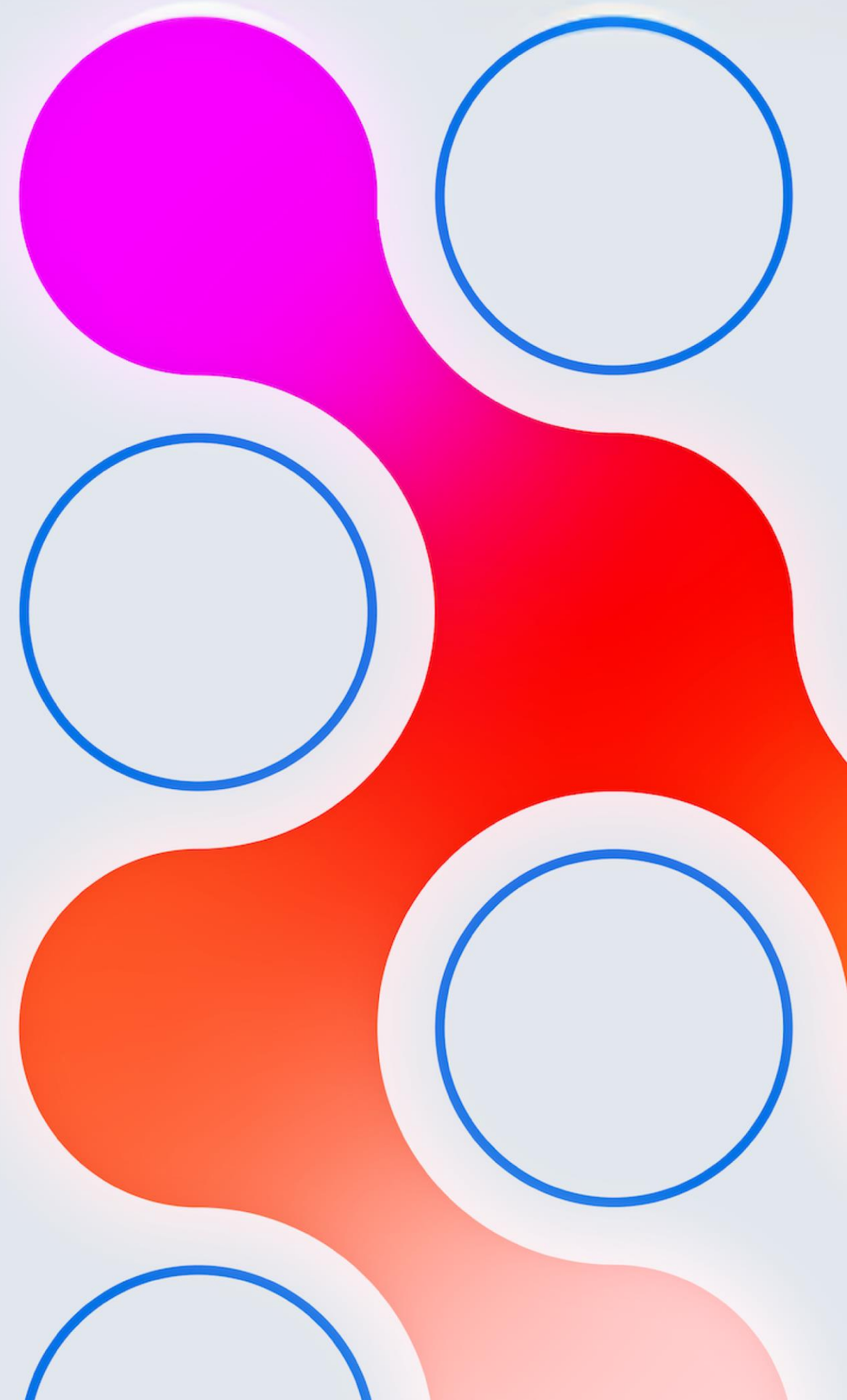
2

Significant parameter errors or reasoning ignores tool outputs in multiple steps.

1

Tool calls are invalid, absent, or reasoning is entirely disconnected from results.

Routing agents and decision patterns



Intent classification and routing architecture

What a routing agent does

A routing agent receives a request. It classifies the intent and directs it to the appropriate specialist agent. Critically, the routing agent does not need to be a full agent with a reasoning loop — a single model call is sufficient for classification.

A good classifier prompt must include:

Clear category definitions

Describe each destination agent in one sentence. Ambiguous descriptions produce ambiguous routing.

Per-category examples

2 to 3 example inputs per category to calibrate the model without fine-tuning.

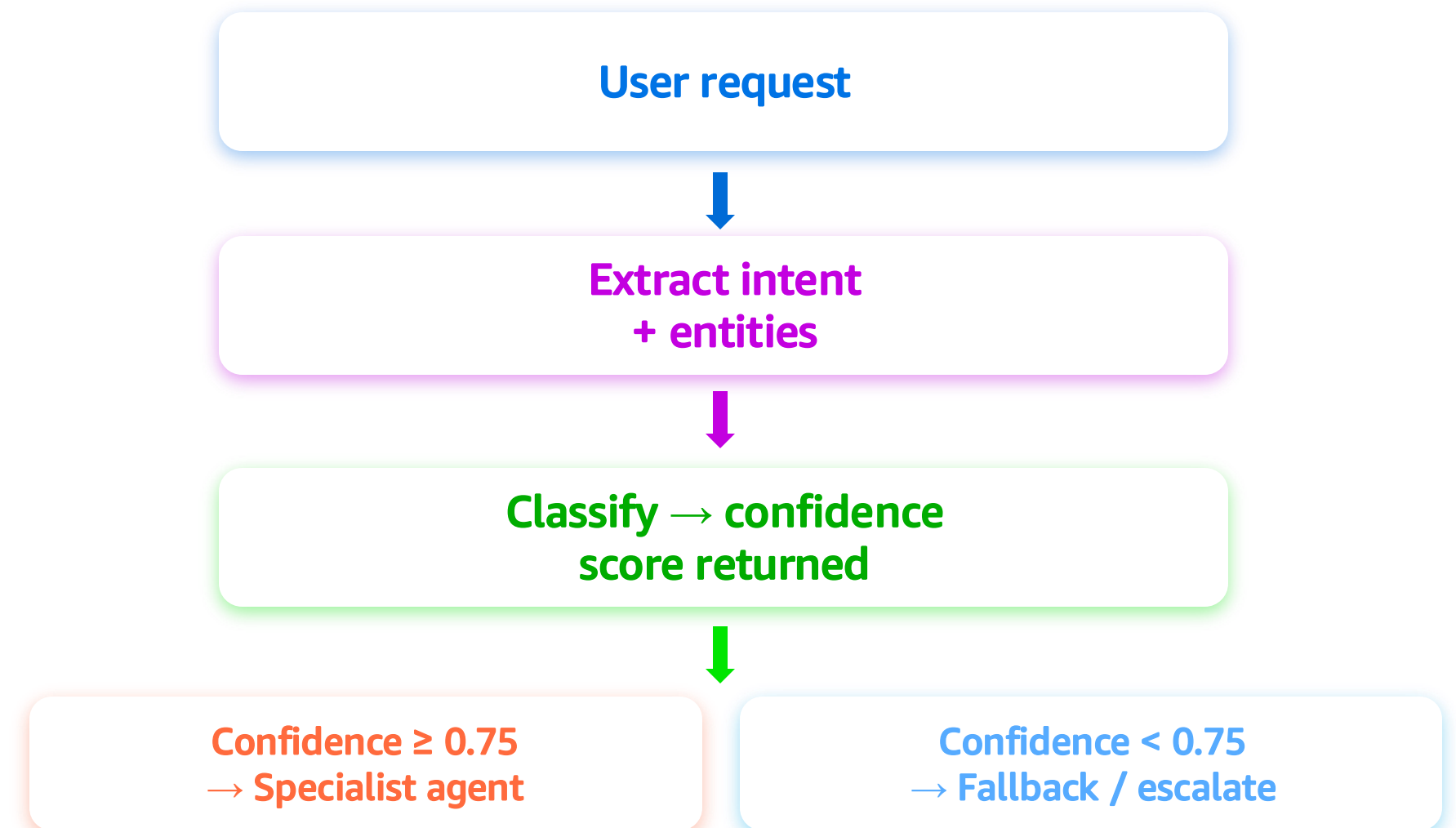
Handling ambiguous inputs

Define an explicit "unknown" or "multi-intent" category and decide in advance what happens to those requests.

Confidence scoring

Ask the classifier to return a confidence score alongside the category. Route low-confidence requests to a fallback path.

Decision flow



Opinion: Do not allow a low-confidence routing decision silently proceed to a specialist agent. A wrong routing decision compounds — the specialist agent wastes tokens and returns a confusing result. Explicit fallback paths save far more time than they cost.

Routing strategy selection: When to use what

Strategy	Accuracy	Latency	Cost	Complexity	When to use it
Keyword rules	Medium	< 1 ms	\$	Low	Narrow, stable input taxonomy with ≤ 10 well-defined categories
★ Recommended start Embedding routing	High	5–15 ms	\$\$	Medium	Semantic variety across intents; rightstarting point for most teams
Classifier model	Very High	10–30 ms	\$\$\$	Medium	High-volume routing (> 10K/day) with > 10 intent categories
LLM routing	Highest	200+ ms	\$\$\$\$	High	Ambiguous multi-intent inputs where classification accuracy is critical
Hybrid (rules+LLM)	Highest	Variable	\$\$\$	High	Production: fast rule path for known patterns; LLM fallback for the rest

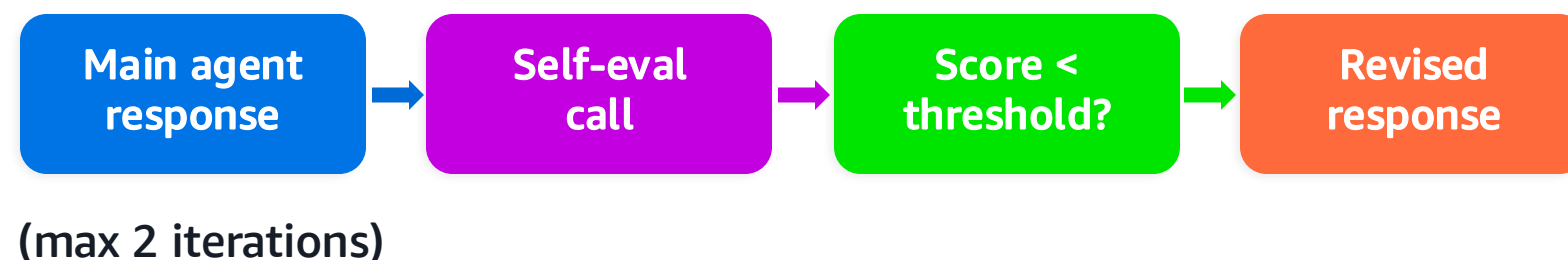


AWS recommendation: Start with embedding-based routing using Amazon Bedrock Knowledge Bases for intent matching. Add an LLM fallback for inputs scoring under your confidence threshold. Only invest in a fine-tuned classifier when you have > 10,000 labeled examples and routing latency is a documented constraint.

The self-correction pattern

How it works

After the main response is generated, a second model call asks: "Does this response fully and correctly address the original goal?" If not, the model produces a revised version. Cap at two self-correction iterations to bound latency.



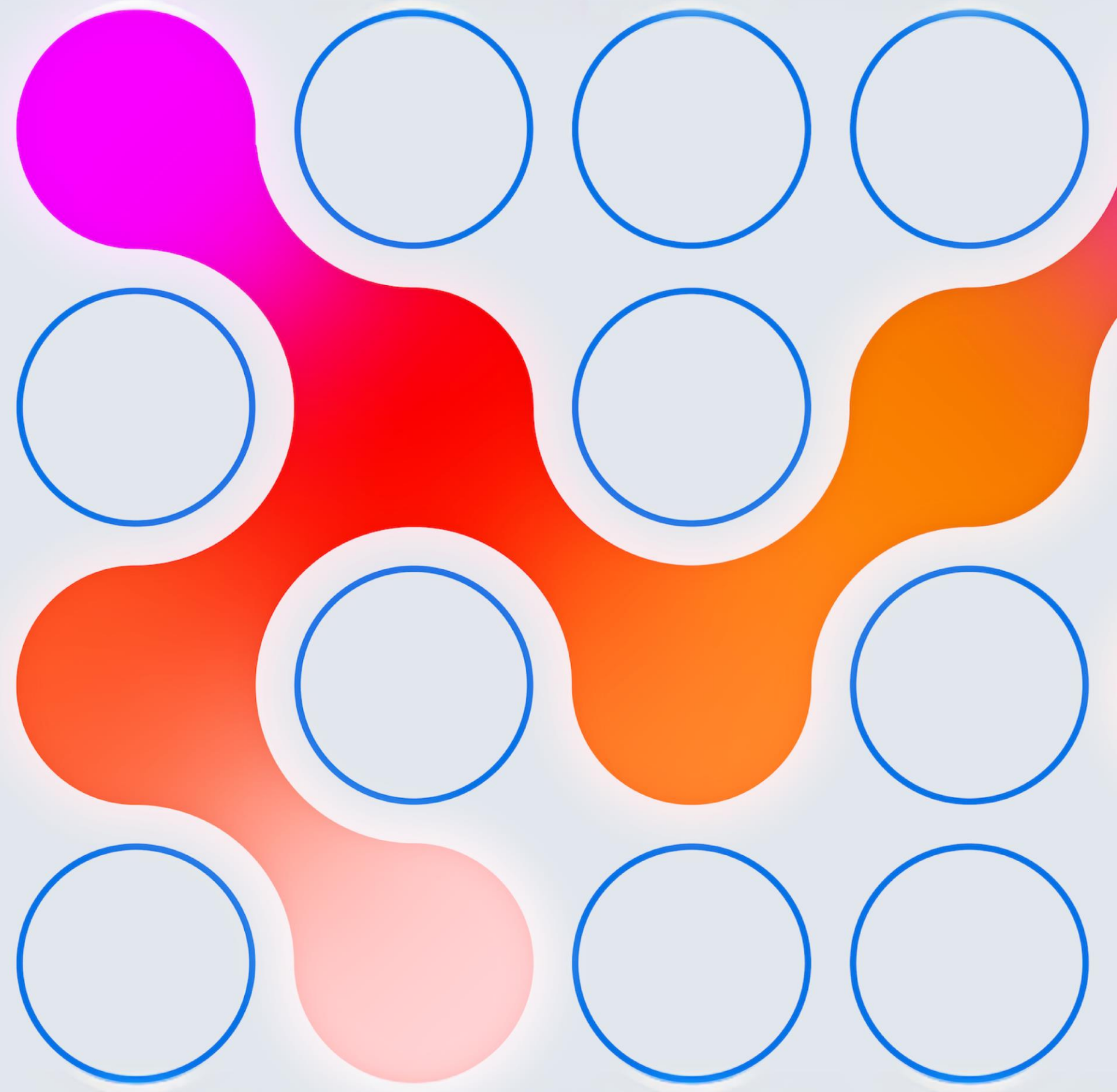
When to use it

- ✓ Agent has enough context to recognize a poor response (e.g., code generation, structured report writing)
- ✓ Task quality justifies +20–40% latency overhead (e.g., critical infrastructure decisions)
- ✗ Conversational agents — users perceive the delay; latency wins over quality here
- ✗ Basic retrieval tasks — if the agent answers from a retrieved fact, self-correction adds cost without benefit

Opinion: Self-correction is often used as a bandage for a weak system prompt.

Before adding self-correction, ask: Could a better system prompt or clearer tool descriptions address the failure mode? In our experience, 70% of cases where teams reach for self-correction are better solved by improving the initial prompt. Self-correction should be a deliberate architectural choice for high-stakes output, not the default response to quality issues.

Evaluation tooling landscape



Evaluation tooling: Capabilities at a glance

Amazon Bedrock AgentCore Evaluations

- Managed on-demand evaluation runs
- Built-in evaluators (fluency, correctness, helpfulness)
- CloudWatch integration—zero infra to manage
- Custom evaluators using Amazon Bedrock model prompts

LangSmith (LangChain)

- Full trace capture + annotation
- Dataset management with versioning
- Built-in LLM-as-judge workflows
- Tight LangChain / LangGraph integration

LangFuse

- Full trace capture and scoring
- LLM-as-judge evaluations built-in
- Self-host for data residency requirements
- Works with any framework, not just LangChain

Weights & Biases

- Experiment tracking across agent versions
- Prompt version management
- Score dashboards and trend analysis
- Strong fit for ML-heavy engineering teams

Patronus AI

- Custom evaluation criteria (define your own)
- Automated regression tracking across versions
- Hallucination and faithfulness scoring
- Enterprise compliance audit trails

Deepchecks

- Automated hallucination detection
- Data drift detection for RAG pipelines
- CI/CD integration — runs as a pipeline step
- Open source core + enterprise features

Tool selection guidance: Choose based on your context

If your primary requirement is...

AWS-native stack, minimize infra overhead	→ Amazon Bedrock AgentCore Evaluations	Zero infrastructure. Scores are published directly to CloudWatch. IAM-controlled.
Using LangChain or LangGraph as your framework	→ LangSmith	Built by the same team. Traces are captured automatically with zero instrumentation beyond a single environment variable.
Data residency requirements (EU, regulated industries)	→ LangFuse (self-hosted)	Open source with a clean Docker deployment. All data stays in your VPC. Comparable features to LangSmith without the SaaS dependency.
You need custom evaluation criteria that evolve with your product	→ Patronus AI	Designed for teams that need to define and maintain their own rubrics. Strong regression tracking.
ML team comfort; you already track experiments	→ Weights & Biases	If your team already uses Weights & Biases for model experiments, extending it to agent eval is low-friction.
RAG pipeline validation or hallucination is your #1 concern	→ Deepchecks	The strongest automated hallucination detection of the group. Integrates as a CI/CD step.

Evaluation anti-patterns: Where teams go wrong



Evaluating only the happy path

A test dataset of only clean, well-formed inputs miss 80% of real failures. Edge cases and adversarial inputs expose the failure modes that matter.



Fix

Deliberately include ambiguous, incomplete, and adversarial inputs in your dataset. Add a new test case every time a production failure occurs.



Using the same model as agent and judge

A model evaluating its own outputs is unreliable. Self-evaluation bias means the judge consistently rates the agent too high.



Fix

Always use a different model as the judge — ideally a more capable one. Claude 4.6 Sonnet as agent + Claude 4.6 Opus as judge is a proven pairing.



Evaluating once, not continuously

An agent that passes evaluation today can regress tomorrow — through prompt changes, model updates, or shifts in the input distribution.



Fix

Run evaluation on every PR that changes the agent. Gate merges on score thresholds. Track scores across agent versions in DynamoDB.



Optimizing for one metric at the expense of others

A prompt change that improves task completion by 5% but doubles cost for each task is not a net improvement.

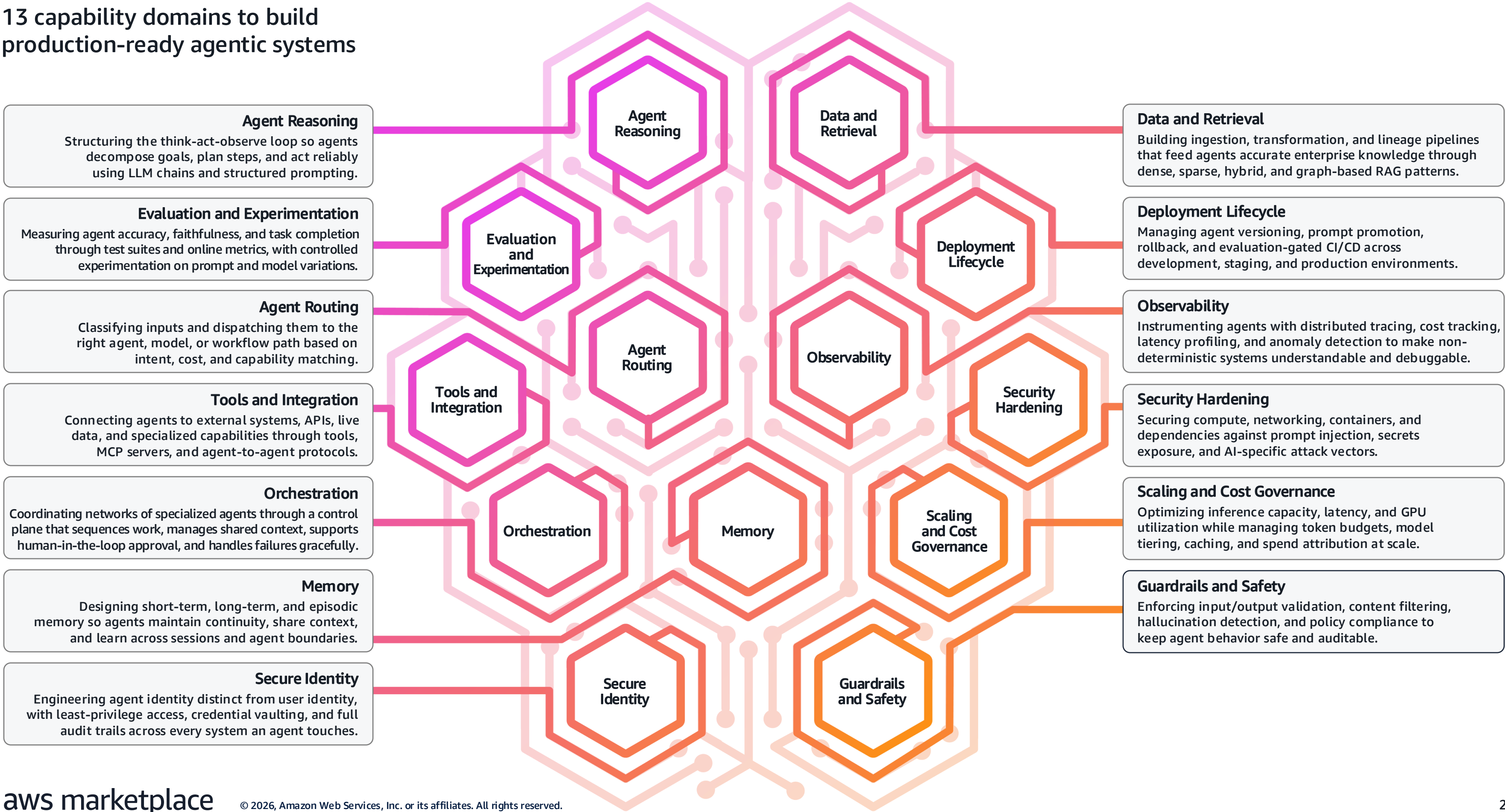


Fix

Define a composite score that weights all four dimensions. Review the full metric set together before approving any agent change.

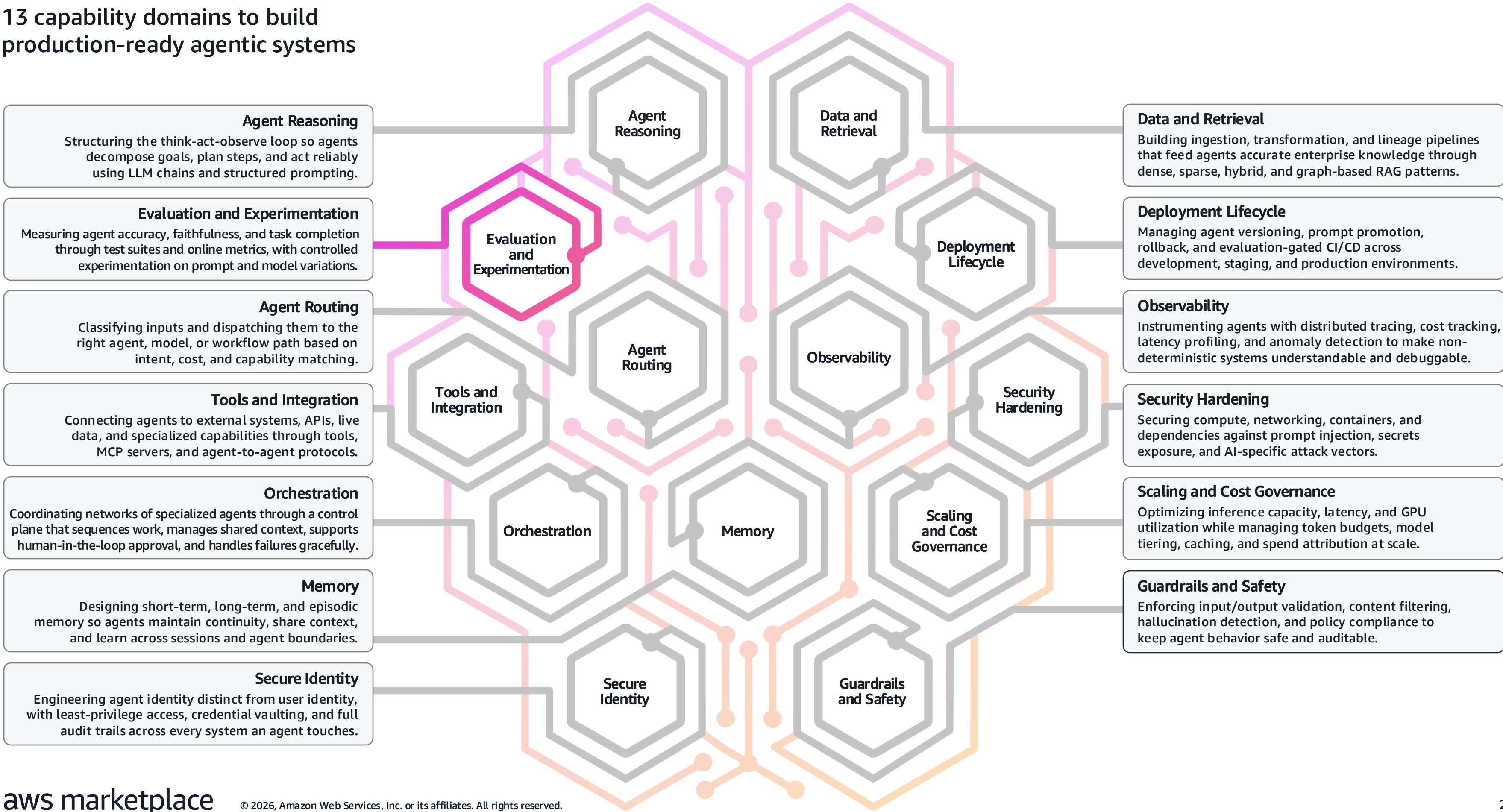
The Agentic AI capability stack

13 capability domains to build production-ready agentic systems

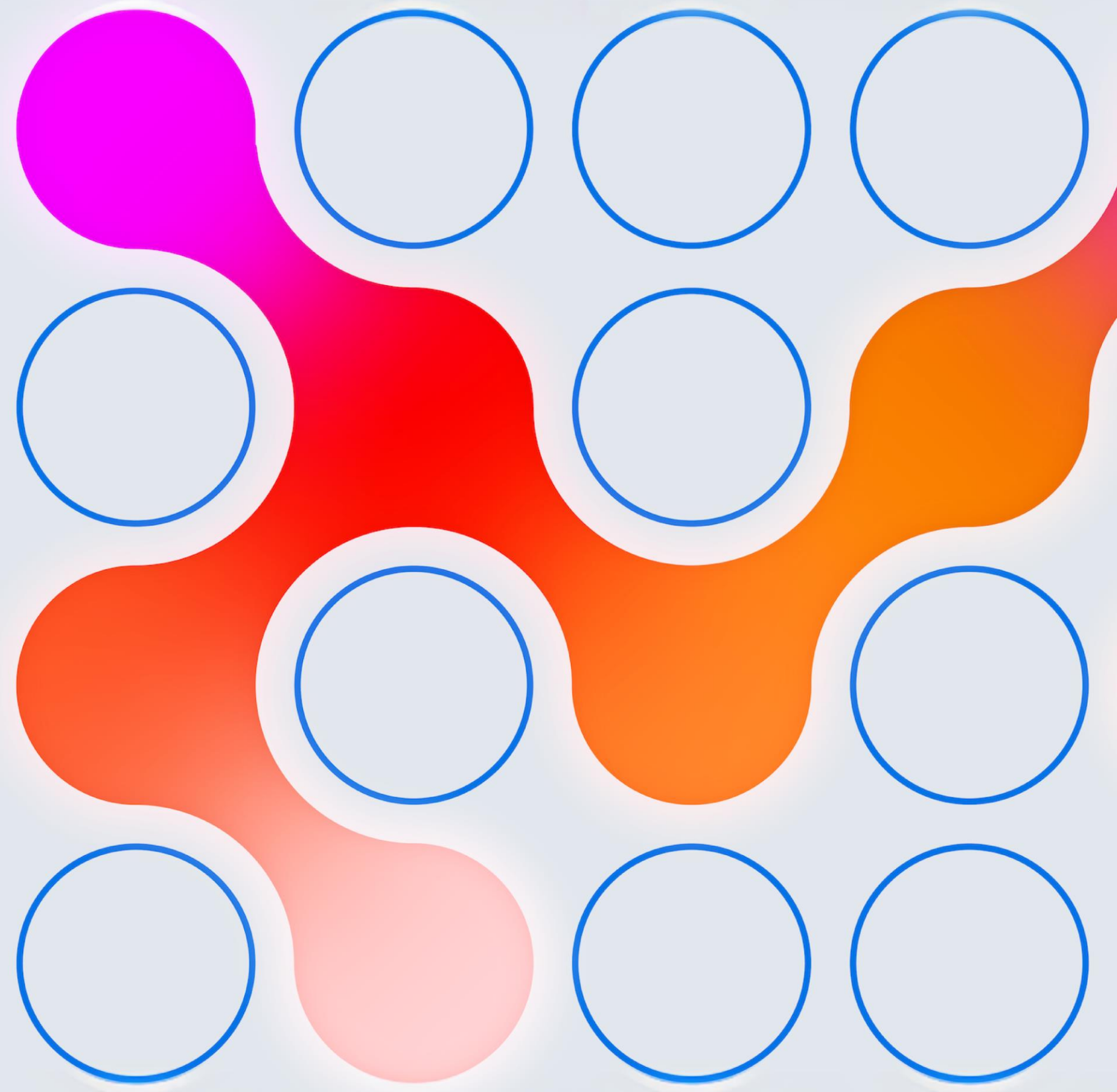


The Agentic AI capability stack

13 capability domains to build production-ready agentic systems



Demo



Agentic AI tools and capabilities in AWS Marketplace

AGENT REASONING

Claude

cohere

Hugging Face

LangChain

Dify

Progress Agentic RAG

composable

Amazon Bedrock

STRANDS AGENTS

EVALUATION & EXPERIMENTATION

LangChain

Langfuse

vellum

Weights & Biases

comet LLM

Patronus AI

deepchecks.

Amazon Bedrock AgentCore (Evaluations)

Amazon SageMaker AI

Amazon CloudWatch

AGENT ROUTING

LangChain

AWS Step Functions

Amazon EventBridge

Amazon Bedrock

TOOLS & INTEGRATION

algolia

mapbox

Deepgram

Kong Connect

Tyk

TAILWINDS

portkey

SMARTBEAR

krakenD

AWS Lambda

Amazon API Gateway

Amazon Bedrock AgentCore (Gateway)

ORCHESTRATION

Temporal

orkes

PREFECT

ASTRONJMER

AWS Step Functions

Amazon Bedrock Agents

Amazon MWAA

MEMORY

zilliz

redis

elastic

TiDB Cloud

MongoDB Atlas

Weaviate

yugabyteDB

Pinecone

Amazon OpenSearch

Amazon MemoryDB

Amazon Bedrock AgentCore (Memory)

SECURE IDENTITY

Ping Identity

okta

CYBERARK

Amazon Cognito

AWS Secrets Manager

Amazon Bedrock AgentCore (Identity)

DATA & RETRIEVAL

CONFLUENT

databricks

Fivetran

Collibra

Alation

Bigeye

MONTE CARLO

boomi

Amazon Bedrock Knowledge Bases

Amazon S3

AWS Glue

DEPLOYMENT LIFECYCLE

Pulumi

GitLab

Vercel

LaunchDarkly

Vault

KNIME

AWS CodePipeline

Amazon SageMaker AI

AWS CDK

OBSERVABILITY

DATADOG

honeycomb.io

arize

LangWatch

True Watch

INSTANA

logz.io

Amazon CloudWatch

AWS X-Ray

AWS Distro for OpenTelemetry

SECURITY HARDENING

snyk

Protect AI

Prompt

tailscale

Amazon GuardDuty

AWS Security Hub

Amazon Inspector

SCALING & COST GOVERNANCE

ClickHouse

NVIDIA AI ENTERPRISE

Modal

cast.ai

baseten

anyscale

hydrolix

StormForge

Amazon EKS

AWS Fargate

Amazon Bedrock AgentCore (Runtime)

GUARDRAILS & SAFETY

Claude

Guardrails AI

Arthur

Amazon Bedrock Guardrails

Amazon Bedrock AgentCore (Policy)

2,800+

Agentic AI tools and capabilities

To help you build, deploy, and operate agentic systems on AWS — available through your AWS account.



© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.

30

Why AWS Marketplace?

Streamline product discovery, procurement, and deployment.



AWS Marketplace offers pricing flexibility

Flexible consumption models

Ease into commitments

Free trials

Get started with a free trial before committing to purchase.



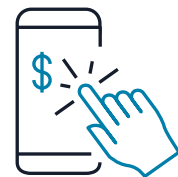
Usage-based subscriptions

Pay for software and compute capacity by the second or hour with no long-term commitments.



Private offers

Negotiate a custom price with a seller or Channel Partner. Accept the offer in AWS Marketplace.



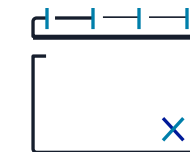
Bring your own license

Migrate to AWS with your existing product licenses. Intended for pre-existing customers.



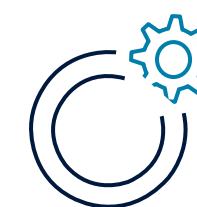
Contracts

Make a monthly, annual, or multi-year payment and receive a discount.



Flexible payment scheduler

Set a custom billing frequency with your seller.



Key takeaways

- Build your test dataset — start with 20 to 50 examples covering your most common and most critical task types
- Instrument your agent with LangFuse or LangSmith to capture full execution traces per task
- Configure Amazon Bedrock AgentCore Evaluations with built-in evaluators and at least one custom rubric
- Add a two-layer scoring step (rule-based first, then LLM-as-judge) to your Step Functions evaluation pipeline
- Gate pull requests: Block any merge where task completion drops under your defined threshold
- Implement embedding-based routing on Amazon Bedrock Knowledge Bases; add LLM fallback for confidence < 0.75



Visit the technical repository

The Agentic DevOps assistant now has a working architecture built with parallel test orchestration, two-layer scoring, and Amazon Bedrock AgentCore integration.

Next steps



Agent evaluation and decision engines

The companion guide covers how to measure agent performance with Amazon Bedrock AgentCore Evaluations, diagnose failures, and apply the decision engines and guardrails.



Building agentic systems on AWS

Explore the 14-module workshop series. Learn to design, orchestrate, and scale multi-agent systems—from foundational patterns to production deployment—with structured tutorials and vetted AWS Marketplace integrations.

Coming up— Module 4

Multi-agent architectures: How multiple specialized agents coordinate as a system, using A2A and MCP protocols, with Amazon Bedrock Multi-Agent Collaboration at the center.

Thank you!

